

IT5008: Tutorial 3 — Simple Queries

Pratik Karmakar

School of Computing,
National University of Singapore

AY26/27 S1



The Book Exchange Schema (Recap)

Same NUN Book Exchange scenario, now in its **refactored** form (department pulled out, available dropped — Tutorial 2's Q4).

- **department**(department, faculty)
- **student**(name, email, year, department (*FK*), graduate)
- **book**(title, format, pages, language, author, publisher, pubdate, isbn10 (*unique*), ISBN13)
- **copy**(owner, book, copy) — FKs to student and book
- **loan**(borrower, owner, book, copy (*FK to copy*), borrowed, returned)

Simple queries only

No **nested** or **aggregate** queries in your answers this tutorial. Focus on **single-table** and **multi-table** joins, plus the set operators UNION/INTERSECT/EXCEPT.

Preliminary: Single-Table Queries

Not discussed in class — attempt before the tutorial, since later questions build on these.

- (a) Print the different departments.
- (b) Print the different departments in which students are enrolled.
- (c) For each copy that has been borrowed and returned, print the ISBN-13 of the book and the duration of the loan. Order by ISBN-13 (ascending), then duration (descending). Use a **single** table.

1(a). Different Departments

```
1 SELECT d.department
2 FROM department d;
```

Why no DISTINCT? department is the **PRIMARY KEY** of the table department — duplicates cannot occur.

1(b). Departments With Enrolled Students

Now we query student: many students can share a department, so we **do** need DISTINCT.

```
1 SELECT DISTINCT s.department
2 FROM student s;
```

Wording matters. This asks for “the different *departments*”. Had it instead asked for “the department of the different *students*”, DISTINCT would be misplaced — students are already unique by email, so the (wrong-question) query would just be:

```
1 SELECT s.department
2 FROM student s;
```

1(c). Loan Duration, Single Table

Only loan is needed. A loan not yet returned has no fixed duration yet — treat it as running until today with COALESCE.

```
1 SELECT l.book,  
2     (COALESCE(l.returned, CURRENT_DATE) - l.borrowed + 1) AS  
   duration  
3 FROM loan l  
4 ORDER BY l.book ASC, duration DESC;
```

Equivalently, with CASE:

```
1 SELECT l.book,  
2     ((CASE WHEN l.returned IS NULL THEN CURRENT_DATE  
3         ELSE l.returned END) - l.borrowed + 1) AS duration  
4 FROM loan l  
5 ORDER BY l.book ASC, duration DESC;
```

Discussion: Multi-Table Queries

To be discussed in class — attempt before the tutorial; you may be called on to present.

- (a) For each unreturned loan of a book published by 'Wiley', print the book's title, the owner's name and faculty, and the borrower's name and faculty.
- (b) **Integrity check.** Print the distinct emails of students who borrowed or lent a copy **before** they joined the university. There should not be any.
- (c) Print the distinct emails of students who borrowed or lent a copy **on the day** they joined.
- (d) Print the distinct emails of students who borrowed **and** lent a copy **on the day** they joined.
- (e) Print the distinct emails of students who borrowed **but did not lend** a copy **on the day** they joined.
- (f) Print the distinct ISBN-13 of books that have **never** been borrowed.

2(a). Unreturned Wiley Loans — via copy

We stitch tables together by joining each **FOREIGN KEY** to the **PRIMARY KEY** it references.

```
1 SELECT b.title,  
2     s1.name AS ownerName,    d1.faculty AS ownerFaculty,  
3     s2.name AS borrowerName, d2.faculty AS borrowerFaculty  
4 FROM loan l, book b, copy c,  
5     student s1, student s2, department d1, department d2  
6 WHERE l.book = b.ISBN13  
7     AND c.book = l.book AND c.copy = l.copy AND c.owner = l.owner  
8     AND l.owner = s1.email AND l.borrower = s2.email  
9     AND s1.department = d1.department  
10    AND s2.department = d2.department  
11    AND b.publisher = 'Wiley' AND l.returned ISNULL;
```

2(a). ...and Why We Should Drop copy

The copy row referenced by loan's (owner, book, copy) already **must** exist — that's exactly what the loan → copy foreign key guarantees. Joining to copy only to discard it is a **redundant table**.

```
1 SELECT b.title,  
2     s1.name AS ownerName,    d1.faculty AS ownerFaculty,  
3     s2.name AS borrowerName, d2.faculty AS borrowerFaculty  
4 FROM loan l, book b,          -- no more copy table  
5     student s1, student s2, department d1, department d2  
6 WHERE l.book = b.ISBN13  
7     AND l.owner = s1.email AND l.borrower = s2.email  
8     AND s1.department = d1.department  
9     AND s2.department = d2.department  
10    AND b.publisher = 'Wiley' AND l.returned ISNULL;
```

You can and should omit tables/columns whose values are already guaranteed by the schema — redundant tables may be penalized.

2(a). ...or With INNER JOIN

```
1 SELECT b.title,  
2     s1.name AS ownerName,    d1.faculty AS ownerFaculty,  
3     s2.name AS borrowerName, d2.faculty AS borrowerFaculty  
4 FROM loan l  
5 INNER JOIN book      b  ON l.book = b.ISBN13  
6 INNER JOIN student  s1 ON l.owner = s1.email  
7 INNER JOIN student  s2 ON l.borrower = s2.email  
8 INNER JOIN department d1 ON s1.department = d1.department  
9 INNER JOIN department d2 ON s2.department = d2.department  
10 WHERE b.publisher = 'Wiley' AND l.returned ISNULL;
```

Convention: put PK–FK join predicates in ON; put everything else in WHERE.

2(b). Integrity Check — Borrowed/Lent Before Joining

Either role (owner or borrower), compared against the student's join date.

```
1 SELECT DISTINCT s.email
2 FROM loan l, student s
3 WHERE (s.email = l.borrower OR s.email = l.owner)
4      AND l.borrowed < s.year;
```

This should return **no rows** — a student cannot lend or borrow before enrolling. (In practice, some rows in the current dataset violate this: a data-quality issue, not a query bug.) `INNER JOIN` or `UNION` work equally well here.

2(c). Borrowed or Lent on Joining Day

Via UNION (dedupes automatically — no DISTINCT needed):

```
1 SELECT s.email FROM loan l, student s
2 WHERE s.email = l.borrower AND l.borrowed = s.year
3 UNION
4 SELECT s.email FROM loan l, student s
5 WHERE s.email = l.owner AND l.borrowed = s.year;
```

Equivalently, via OR (needs DISTINCT):

```
1 SELECT DISTINCT s.email
2 FROM loan l, student s
3 WHERE (s.email = l.borrower OR s.email = l.owner)
4 AND l.borrowed = s.year;
```

2(d). Borrowed *and* Lent on Joining Day

Via INTERSECT:

```
1 SELECT s.email FROM loan l, student s
2 WHERE s.email = l.borrower AND l.borrowed = s.year
3 INTERSECT
4 SELECT s.email FROM loan l, student s
5 WHERE s.email = l.owner AND l.borrowed = s.year;
```

Equivalently, without INTERSECT — two aliases of loan, one fact per role:

```
1 SELECT DISTINCT s.email
2 FROM loan l1, loan l2, student s
3 WHERE s.email = l1.borrower AND l1.borrowed = s.year
4      AND s.email = l2.owner   AND l2.borrowed = s.year;
```

2(e). Borrowed *but Not* Lent on Joining Day

```
1 SELECT s.email FROM loan l, student s
2 WHERE s.email = l.borrower AND l.borrowed = s.year
3 EXCEPT
4 SELECT s.email FROM loan l, student s
5 WHERE s.email = l.owner AND l.borrowed = s.year;
```

Unlike 2(d), there is **no simple-query alternative** here — “but not” needs EXCEPT (or, outside this tutorial’s rules, a nested/aggregate query).

2(f). Books Never Borrowed

Via EXCEPT:

```
1 SELECT b.ISBN13 FROM book b
2 EXCEPT
3 SELECT l.book FROM loan l;
```

Via LEFT OUTER JOIN — unmatched books get NULL for l.book, so we filter on that:

```
1 SELECT b.ISBN13
2 FROM book b LEFT OUTER JOIN loan l ON b.ISBN13 = l.book
3 WHERE l.book ISNULL;
```

There happens to be no such book in the current dataset — the query must still be correct for *any* dataset satisfying the schema.

Guidelines & Marking Tips

- **No hardcoding.** A query must work on *any* dataset consistent with the schema — we test against other datasets too.
- **Constants only if the question names one.** 'Wiley' is fine because 2(a) says so; don't introduce constants of your own.
- **Readable style.** Alias every table, qualify every column, and uppercase SQL keywords.
- **Set operators dedupe already.** UNION/INTERSECT/EXCEPT remove duplicates automatically — a DISTINCT on top of one is redundant.
- **Partial marks** require: (i) the query executes, (ii) columns are correct (names, types, order), and (iii) row differences are minimal. A query that fails to execute may receive **0**.

Questions?

Drop a mail at: pratik.karmakar@u.nus.edu